autolisp-benchmark — User Manual

clautolisp

July 2, 2026

Contents

1 Introduction

`autolisp-benchmark` is a pure-AutoLISP benchmark suite. You load it into an AutoLISP engine — clautolisp, BricsCAD, or AutoCAD — and it runs a fixed set of workloads, each for about five seconds, then prints how much work each one completed. Because the suite is portable AutoLISP and detects the engine and clock automatically, the numbers from two engines on the same hardware are directly comparable.

2 Running the suite

2.1 Under clautolisp directly

```
clautolisp -l autolisp-benchmark/harness/run.lsp \
           -x '(autolisp-benchmark-run-all)'
```

`run.lsp` loads the harness modules and every workload, then defines `(autolisp-benchmark-run-all)`. Loading it does **not** start a run — you invoke the run function yourself (or the Makefile does it for you).

2.2 Via the Makefile

```
make -C autolisp-benchmark run                  # clautolisp, 5 s / class
make -C autolisp-benchmark run BENCH_SECONDS=10 # 10 s / class
make -C autolisp-benchmark run-clautolisp-ccl   # CCL image instead of SBCL
make -C autolisp-benchmark test                 # fast smoke run (0.2 s/class)
```

The `run` targets rebuild the `clautolisp` executable first if any runtime source is newer than the binary, so you never measure a stale build.

2.3 Inside a real CAD engine (BricsCAD / AutoCAD) via alfe

```
make -C autolisp-benchmark run-bricscad-macos
make -C autolisp-benchmark run-bricscad-windows
make -C autolisp-benchmark run-autocad-windows
```

These drive the benchmark inside the CAD process through `alfe`'s file-IPC backend. They are gated on the host OS (running a macOS-only target on Linux prints a `[skip]` line and exits 0), and they require the corresponding CAD engine to be installed and reachable by alfe.

You can also drive alfe by hand:

```
alfe --bricscad \
     -x '(setq *bench-seconds* 5.0)' \
     -l autolisp-benchmark/harness/run.lsp \
     -x '(autolisp-benchmark-run-all)'
```

3 Configuration

Set these AutoLISP globals before calling `(autolisp-benchmark-run-all)` (the Makefile sets them for you from `make` variables):

- `*bench-seconds*` — wall-clock budget per class. Default `5.0`.
- `*autolisp-benchmark-root*` — directory holding `harness/` and `benchmarks/`. Default `"autolisp-benchmark/"` (relative to the process working directory).
- `*bench-gc-interval*` — how often the memory-gc workload calls `(gc)`. Default `25`.

Makefile variables:

- `BENCH_SECONDS` — passed to `*bench-seconds*` for `run` targets.
- `BENCH_SMOKE_SECONDS` — budget for the `test` smoke run. Default `0.2`.

4 Reading the report

A run prints a banner, one row per class, and a summary:

```
======================================================================
  AutoLISP benchmark suite
  implementation : clautolisp 1.3.1
  platform       : clautolisp
  clock source   : MILLISECS (integer ms)
  seconds/class  : 5.0
  benchmarks     : 7
======================================================================
class         benchmark                  iters       iters/sec         us/iter
arithmetic    arithmetic              ...
...
----------------------------------------------------------------------
  total iterations : ...
  total time (ms)  : ...
  overall iters/s  : ...
======================================================================
```

- **implementation** comes from `(ver)` — the reliable cross-engine self-identification.
- **clock source** is the timer actually used: `MILLISECS` when it is live, otherwise `DATE` scaled to milliseconds.
- **iters/sec** (higher is faster) is the figure to compare across engines; every class runs for roughly the same wall-clock time, so the elapsed column is nearly constant by construction and it is the throughput that distinguishes implementations.
- **us/iter** is the inverse view: microseconds of work per iteration (lower is faster).

To compare engines, run the suite on each on the same machine with the same `BENCH_SECONDS` and compare the per-class `iters/sec`.

5 Running a single class

```
(autolisp-benchmark-run "strings")   ; run just the strings workload
```

6 Adding a workload

1. Create `benchmarks/<name>.lsp`. Define a function `(bench-<name> reps)` that performs `reps` units of work in a tight loop and returns something (the return value is ignored). Keep any one-time setup outside the loop.
2. End the file with `(defbench "<name>" "<class>" 'bench-<name>)`.
3. Add the file path to `*autolisp-benchmark-files*` in `harness/manifest.lsp` (AutoLISP has no portable directory walker, so the file list is explicit — same convention as autolisp-test).

Use only operators available on every target you intend to compare. In particular, prefer entity primitives over `(command …)`, which clautolisp does not implement.

7 Caveats

- The `memory-gc` workload forces full collections with `(gc)`; on a real CAD host `(gc)` may be lighter or a hint, so treat cross-engine GC numbers as indicative rather than exact.
- The `file-io` workload writes a single scratch file in the platform temp directory (`$TMPDIR` / `$TEMP` / `$TMP`, else `/tmp`) and reuses it; it leaves one small file behind.
- Compiled-code (`vlisp-compile` / `.fas`) comparison is out of scope on clautolisp (no compile step); see `PLAN.md` for the CAD-only plan.